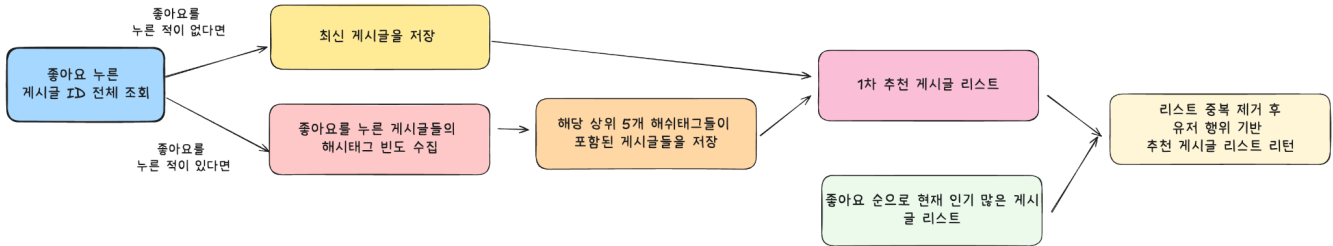


1. 쿼리 튜닝을 통한 유저 게시물 추천 API 성능 개선

데이터 플로우



- **비효율적인 데이터 조회:** 추천 후보군을 선정할 때, 처음부터 무거운 전체 데이터(Post 객체)를 조회하여 과도한 메모리 사용 및 DB 부하를 유발.
- **N+1 쿼리 문제:** 각 게시물의 '좋아요' 여부, 댓글 수 등 추가 정보를 가져오는 과정에서 반복적인 쿼리가 실행되어 응답 지연의 원인이 되었음.
- **정적인 추천 결과:** 사용자에게 항상 동일한 순서의 추천 목록을 제공하여, 추천의 신선도와 사용자 경험(UX)이 저하.

해결 과정

- **ID 기반 조회 전략 도입:** Post 객체 대신 **가벼운 Post ID 목록을 먼저 조회**하도록 쿼리 전략을 변경. 이를 통해 메모리에서 다루는 데이터의 양을 최소화하고 필터링 속도를 향상.
- **일괄 처리(Batch)를 통한 N+1 문제 해결:** 최종적으로 선정된 ID 목록을 IN (...) 절에 담아, 필요한 모든 게시물 정보, 좋아요 상태, 댓글 수를 **단 몇 번의 쿼리만으로 일괄 조회**하도록 최적화.
- **추천 결과 셔플링(Shuffle) 적용:** 최종 추천 후보 목록을 사용자에게 반환하기 직전, Collections.shuffle()을 이용해 무작위로 섞어주는 로직을 추가.

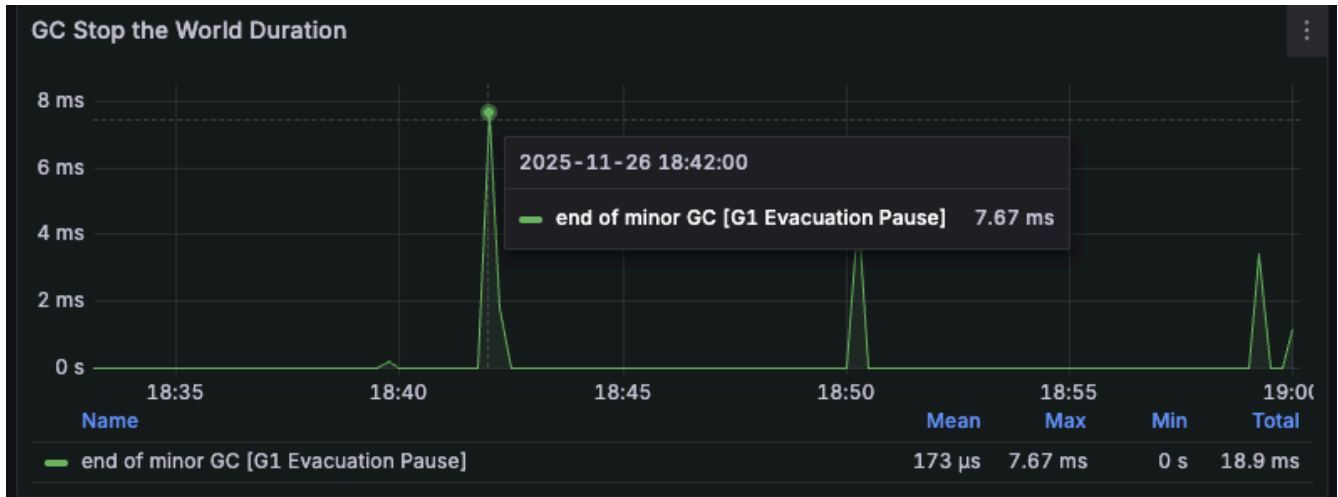
결과

- **사용자 경험(UX) 혁신:** 추천을 요청할 때마다 새롭고 다채로운 게시물 목록을 제공함으로써 사용자의 서비스 만족도와 콘텐츠 소비율을 상승.
- **정량적 성능 개선 (부하 테스트 결과):** ApacheBench(ab)를 사용하여 동시 사용자 50명, 총 요청 500회 조건으로 부하 테스트를 진행한 결과, 시스템 처리량과 안정성이 대폭 개선.

지표	개선 전	개선 후	개선율
초당 요청 처리량 (RPS)	1.98 req/s	6.08 req/s	약 3.1배 향상
평균 응답 시간 (동시 요청)	506.076 ms	164.364 ms	약 3.1배 단축
99% 요청 응답 시간	48,054 ms	16,711 ms	약 2.9배 단축
실패/오류 요청	18 (Non-2xx)	0	오류 제거

2.추천 API 성능 개선을 위한 JVM 및 GC 튜닝 적용

모니터링



문제 원인

- **간헐적인 응답 지연 (Tail Latency):** 쿼리 튜닝 후 처리량(RPS)은 개선되었으나, 프로파일링 결과 GC(Garbage Collection) 수행 시 **Stop-the-world 현상이 길게 발생**하여 간헐적으로 응답 시간이 증가하는 문제가 발생.
- **비정상적인 메모리 인식:** Docker 컨테이너 환경에서 메모리 제한 설정이 미비하여 Host OS의 메모리를 잘못 인식하고 있었으며, 이로 인해 Heap 영역이 부족해 빈번한 GC가 발생. (Pause Time: 최대 ~3.47초)

해결 과정

- **메모리 가시성 확보:** 컨테이너 환경에서 JVM이 할당된 메모리 제한을 정확히 인식하도록 **-Xms1536m -Xmx1536m** 옵션을 설정.
- **GC 알고리즘 검토 (Latency 최적화):** 낮은 지연 시간(Latency)에 유리하다고 알려진 **ZGC (-XX:+UseZGC)** 적용을 테스트하며 메모리 및 GC 설정을 최적화. (DB 튜닝으로 객체 생존 주기가 짧아져 G1GC로도 안정적이었지만, 설정을 명확히 함으로써 안정성 확보)
- **배포 환경 설정 반영:** DockerFile 및 Github Action workflow에 **메모리 제한 및 GC 튜닝 옵션**을 추가하여 배포 환경 전반에 걸쳐 설정을 명확하게 적용

결과

- **시스템 안정성 확보:** 메모리 설정의 명확화 및 GC 최적화를 통해 **시스템 안정성**을 확보.
- **간헐적 지연 현상 해결:** GC로 인한 중단 시간(Stop-the-world)이 0초에 수렴함으로써, 간헐적으로 발생하던 **응답 지연 현상(Tail Latency)**이 근본적으로 해소.
- **안정적인 고성능 환경 구축:** DB 튜닝과 더불어 WAS 레벨의 튜닝까지 완료하여 **부하 상황에서도 안정적인 성능을 유지**할 수 있는 기반을 마련.

3.Audit Log 조회 API 성능 최적화 - 커서 기반 페이징 & 인덱싱 적용

데이터 플로우



문제 원인

- Audit Log ↔ Audit Log Detail 테이블이 1:N 관계로 설계되어 있었음.
- 1,000만 건 이상 로그가 누적된 상태에서 **마지막 페이지 조회 요청 시 4,538ms**의 지연 발생.
- **OFFSET/LIMIT 기반 페이징**은 대량 데이터일수록 불필요한 Full Scan을 유발 → 마지막 페이지 조회 시 성능 저하 극심.
- 커서 탐색 컬럼에도 인덱스가 없어 탐색 비용이 기하급수적으로 증가.

해결 과정

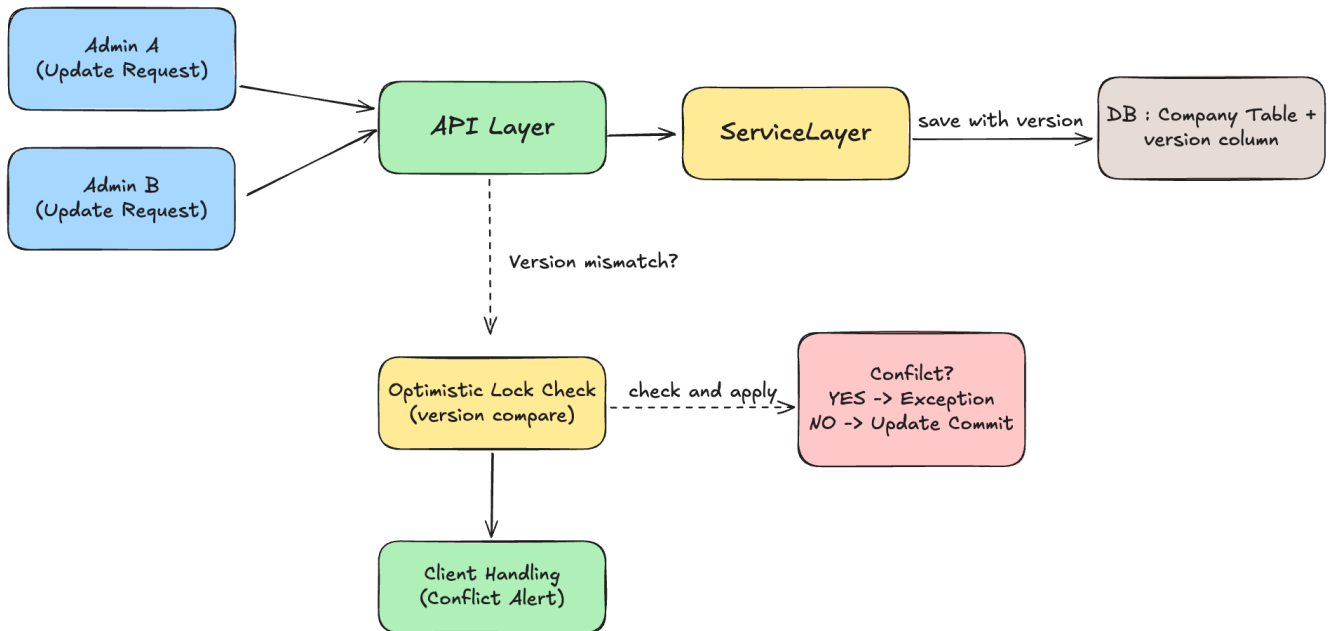
- **커서 기반 페이징 도입**: OFFSET 대신 “마지막 조회 위치”를 기준으로 데이터를 이어서 가져오도록 설계.
→ 필요 없는 앞 페이지 스캔 제거.
- **인덱스 추가 최적화**: 커서 페이징에 사용되는 주요 컬럼을 인덱스로 지정하여, 탐색 시 Random Scan 최소화.
- **쿼리 플랜 검증**: 인덱스 적용 후 실행 계획을 분석, Table Scan → Index Seek로 변경된 것을 확인.

결과

- 응답 속도 99% 단축: **4,538ms → 23ms**로 개선.
- 대용량 로그 확장성 확보: 수천만 건 단위 데이터에서도 안정적 성능 유지.
- 관리자 UX 혁신: 지연 없는 로그 조회로 실시간 감사 / 모니터링 환경 구현.

4. 회사 정보 수정 시 동시성 문제 해결 - 낙관적 락(Optimistic Lock) 적용

데이터 플로우



문제 원인

- 다수의 관리자가 동시에 회사 정보 수정 요청 시 발생하는 Race Condition.
- “조회 → 비즈니스 로직 처리 → 저장” 과정에서 버전 검증 부재로, 마지막 요청만 DB에 반영.
- 그 결과, 앞선 사용자의 수정 내용이 silently overridden(조용히 덮어쓰기) 되는 문제 발생.

해결 과정

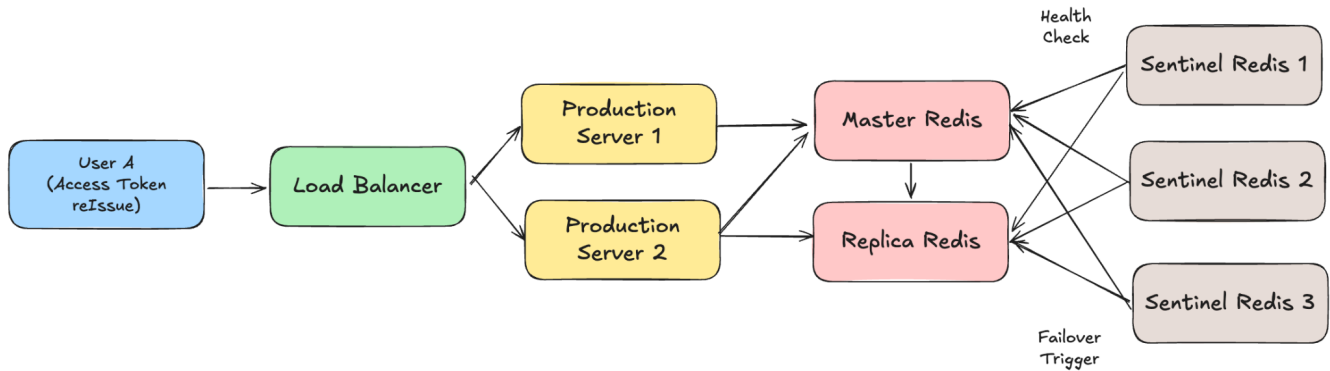
- 버전 관리 컬럼(version) 추가: 엔티티에 버전 번호를 두어 수정 시점의 상태를 추적.
- 낙관적 락 적용:
 - 수정 요청 시 클라이언트에서 전달한 version과 DB의 현재 version을 비교.
 - 불일치 시 예외 발생 → 충돌 감지.
- 재시도 전략 도입: 충돌 시 사용자에게 알림 제공 및 수정 재시도 프로세스 설계.

결과

- 데이터 무결성 보장: 동시에 여러 요청이 발생해도 수정 손실 없음.
- 안정적 다중 사용자 환경 확보: 기업 고객 환경에서도 신뢰성 있는 데이터 관리 가능.
- 비즈니스 신뢰도 강화: 관리자가 안심하고 동시에 데이터를 수정할 수 있는 기반 마련.

5. 분산 운영 서버 환경에서 Refresh Token 불일치 문제 해결 - Redis Sentinel 도입

전체적인 아키텍처



문제 원인

- 초기에는 분산 운영서버 각각의 Redis 컨테이너에 존재하는 RefreshToken 동기화를 위해 **Redis Master-Slave 복제 구조**만 운영.
- 하지만 Master 장애 발생 시, Slave가 자동 승격되지 않아 **데이터 조회 불가** → **서비스 전체 중단** 사태 발생.
- 특히 Refresh Token은 **인증·인가의 핵심 요소**라, 불일치/유실 발생 시 로그인 세션 전체가 무효화됨.

해결 과정

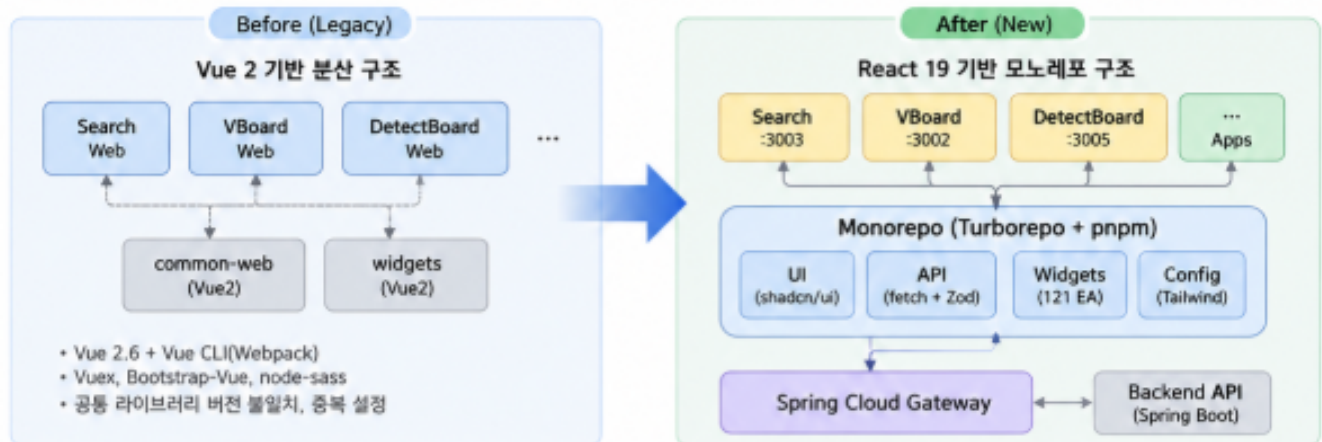
- **Redis Sentinel 구성:**
 - Master 장애 발생 시 Sentinel이 자동 감지.
 - Slave를 즉시 Master로 승격(Automatic Failover) → 무중단 장애 복구 가능.
- **헬스체크 & 알림 시스템** 연계: 장애 상황을 즉시 감지하고 운영팀이 대응할 수 있도록 모니터링 강화.
- **Failover 테스트 시나리오** 수차례 진행하여 실제 운영 환경에서 무리 없이 전환되는지 검증.

결과

- **서비스 연속성 확보:** Master 장애 발생에도 무중단 로그인/세션 유지.
- **안정적 고가용성 환경 구축:** Redis 단일 장애로 인한 서비스 중단 리스크 제거.
- **운영 효율성 향상:** 장애 감지 및 복구 프로세스를 자동화하여 운영 인력의 개입 최소화.

6. 레거시 Vue2 기반 프론트엔드 최신 react 19 기반 마이그레이션

아키텍처 전환



문제 원인

- Vue 2 기반 다수의 마이크로프론트엔드가 개별 프로젝트로 관리되어 공통 라이브러리 버전 불일치 및 설정 중복 발생.
- Vuex, Bootstrap-Vue, node-sass 등 레거시 의존성과 테스트 부재로 유지보수 및 기능 확장이 어려운 구조.
- 운영 중인 대규모 SIEM 서비스 특성상 전체 시스템을 한 번에 교체하기 어려움.

해결과정

- React 19 + TypeScript + Vite 기반 마이그레이션 진행.
- Turborepo + pnpm workspace를 적용해 UI / API / Widgets / Config 공통패키지를 모노레포로 통합.
- Search 앱을 파일럿으로 전환하여 TanStack Query, Zustand, Zod, Tailwind, shadcn/ui 기반 공통 개발 패턴 수립.
- Thin Shell + 독립 앱 구조를 적용하여 기존 Spring Cloud Gateway 기반 배포 구조는 유지하면서 점진적으로 교체.
- Search → VBoard → Log Collection → DetectBoard 순으로 마이그레이션하여 운영 서비스와 신규 개발을 분리.

결과

- 모노레포 및 공통 UI 패키지 구축 및 마이그레이션 완료
- 공통 API·상태관리·UI 패턴을 표준화하여 이후 앱의 반복적인 마이그레이션 비용을 줄일 수 있는 기반 확보.
- 기존 서비스를 중단하지 않고 React 기반 신규 앱을 순차 배포할 수 있는 점진적 전환 구조 확립.

7. SIEM BFF 백엔드 성능 개선 및 리팩토링

문제 원인

- 요청마다 WebClient 커넥션 풀이 생성되고 내부 기능 간 Self-HTTP 호출이 존재하여 **불필요한 네트워크·리소스 비용 발생**.
- 위젯 요청 과정에서 세션이 반복 생성되어 **트래픽 증가 시 세션이 급증하는 문제 발생**.
- JWT 반복 파싱, 블로킹 캐시, 인증 단계 외부 호출 등으로 **요청 처리 경로가 복잡하고 비효율적**.
- Export·인증·파일 처리 로직이 중복되고 일부 Stream이 해제되지 않아 **유지보수 비용 및 리소스 누수 위험 존재**.

해결과정

- WebClient를 **공유 Connection Pool** 구조로 변경하고 Connect / Response Timeout 적용.
- 위젯 Self-HTTP 호출을 **Service 직접 호출 방식**으로 전환하여 네트워크·인증·세션 오버헤드 제거.
- 주요 캐시를 **Non-blocking 처리**로 변경하고 MITRE ATT&CK 등 반복 조회 데이터 캐싱 적용.
- JWT 파싱 최소화, 인증 경로 외부 HTTP 제거, Hazelcast·JWT 기반 **세션 만료 정책 정비**.
- Excel / PDF / CSV Export 로직 공통화 및 **Stream Leak·임시파일 관리 문제 해결**.
- 중복 인증·검증·업로드·다운로드 로직 공용화 및 미사용·Deprecated 라이브러리 제거.

결과

- 외부 API 및 내부 서비스 호출 구조를 단순화하여 **불필요한 Connection·HTTP·Session 생성 감소**.
- 세션 폭증 문제를 해결하고 인증 및 캐시 처리 구조를 최적화하여 **서비스 안정성 향상**.
- Stream 누수 및 임시파일 관리 개선으로 **장시간 운영 환경의 리소스 안정성 확보**.
- 공통 모듈화와 의존성 경량화를 통해 **코드 중복 감소 및 유지보수성 향상**.

8. 온프레미스 SIEM 보안 관제 AI 플랫폼 구축

기술 스택

Python, FastAPI, LangGraph, Pydantic, Elasticsearch, PostgreSQL, SQLite, Qdrant, Ollama/LM Studio, JS, SSE

개발 배경 및 과제

- 보안 관제 데이터가 SIEM의 로그·탐지 결과·룰·티켓·CTI 등에 분산되어 있어, 담당자가 여러 화면과 쿼리를 오가며 직접 조사해야 했음.
- 범용 LLM을 보안 분석에 바로 적용할 경우 숫자·IP·위협 여부를 잘못 생성하거나, 조회 실패를 “이상 없음”으로 해석할 수 있는 위험이 존재했음.
- 다양한 벤더의 로그 포맷과 비정형 정의서로 인해 신규 장비 연동 시 Grok 패턴 작성, 필드 매핑, 규칙서 생성에 많은 수작업이 필요했음.
- 폐쇄망 환경과 보안 제품의 특성상 외부 서비스 의존을 최소화하고, AI가 직접 차단·등록·종결하지 않는 안전한 운영 구조가 필요했음.

핵심 설계 및 구현

- SIEM 데이터를 **7개 보안 신호 축으로 분석**하여 전일 데이터와 직전 7일 기준선을 비교하고, **조치가 필요한 항목을 자동 생성하는 일일 브리핑 엔진 구현**.
- 사용자 질문을 요구사항 단위로 분해하고 **9개 보안 조회 도구를 선택적으로 호출하는 Goal-driven AI Agent 구현**. 도구 예산, 중복 호출 차단, 완료 조건을 적용해 불필요한 조회를 제한.
- 조회 결과를 구조화된 Fact로 관리하고, 최종 답변의 숫자·IP·날짜·MITRE ID를 원본 근거와 재검증하는 Grounding Pipeline 구축.
- Qdrant에 로그 포맷 카탈로그를 적재하고, Dense Embedding과 BM25 결과를 RRF로 결합한 **Hybrid RAG 검색 구현**.
- LangGraph 기반 12개 전문 노드를 구성하여 로그 식별·해석·비교·필드 매핑·파싱 진단·패턴 검증·포맷 생성 요청을 의도별로 처리.
- 샘플 로그의 구조 분석부터 클러스터링, Grok 생성, 재매칭 검증, 표준 필드 매핑, 검수 및 규칙 XLSX 출력까지 자동화.

기술적 의사결정

- “수치는 코드가 계산하고, LLM은 의미만 해석한다”는 원칙으로 결정론적 분석 계층과 생성형 AI 계층을 분리.
- 정상적인 조회 결과 0건과 시스템 조회 실패를 별도 상태로 모델링하여, 장애 상황이 “이상 없음”으로 잘못 표시되지 않도록 설계.
- SIEM의 SQL 연결을 Read-only로 강제하고, **AI는 티켓·룰·오답 처리 초안까지만 생성**하도록 Human-in-the-loop 적용.
- 조사 과정의 도구·인자·결과·근거·종료 사유를 Trace로 저장하고, SSE를 통해 분석 진행 상황을 실시간 제공.
- SQLite WAL, 명시적 트랜잭션, 파일 권한 제한, 데이터 보존 정책을 적용하여 온프레미스 장기 운영 안정성 확보.

결과

- SIEM 데이터 조회부터 분석·보고서 작성까지 하나의 웹 서비스로 통합하여 **반복적인 관제 조사 업무를 단순화**.
- 근거 검증과 결정론적 계산을 통해 LLM이 숫자나 보안 개체를 임의로 생성하는 위험을 구조적으로 제한.
- 수작업 중심이던 로그 클러스터링·Grok 작성·필드 매핑·XLSX 규칙서 생성을 자동화하여 **신규 포맷 초안 생성 시간을 수일 단위에서 수 초 단위로 단축**.
- Golden Set과 회귀 평가 체계를 구축하여 Agent 라우팅, 답변 근거성, 로그 식별 및 포맷 생성 품질을 반복 측정할 수 있도록 개선.